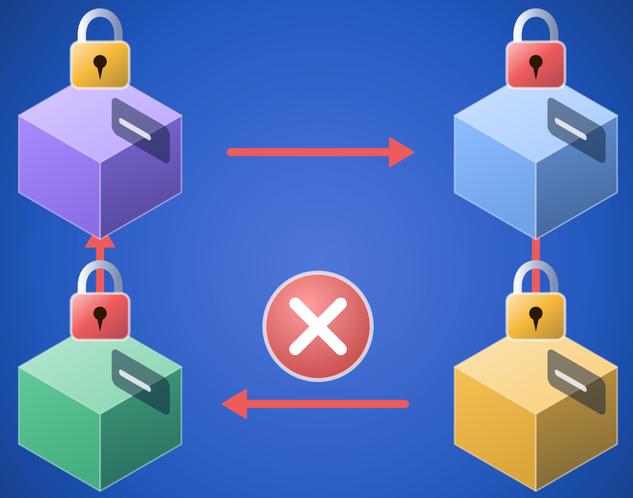
Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI4OS · OPERATING SYSTEMS · WEEK 11

Deadlock

When a set of threads wait for each other without end: how to recognise it, prevent it, avoid it, detect it and recover from it.



Heng Rathpisey · Department GIC · Academic year 2025–2026

Learning objectives

1. Show how deadlock can occur when mutex locks are used.
2. State the four necessary conditions for deadlock.
3. Identify a deadlock in a resource-allocation graph.
4. Evaluate the approaches to preventing deadlocks.
5. Apply the banker's algorithm for avoidance, and the detection algorithm.
6. Evaluate ways of recovering from deadlock.

THIS WEEK

Topic 9

In Week 9 two processes each held one semaphore and waited for the other's. This week we study that situation in general.

DISCUSSION

Four cars arrive at a crossroads at the same time

Each waits for the car on its right to go first. Nobody moves. Whose fault is it, and what rule would have prevented it?

Week 11 · 03

PART 1

The system model and deadlock

A finite number of resources, and threads that compete for them.

Week 11 · 04

The system model

Resource types

R1, R2, … Rm: CPU cycles, memory, files, I/O devices, mutex locks, semaphores.

Instances

Each resource type has a number of identical instances. Any instance of the type satisfies a request.

Three steps

A thread uses a resource in a fixed sequence: request, use, release.

Deadlock with two mutex locks

```
/* thread one */
pthread_mutex_lock(&first);
pthread_mutex_lock(&second);
/* work */
pthread_mutex_unlock(&second);
pthread_mutex_unlock(&first);

/* thread two */
pthread_mutex_lock(&second);
pthread_mutex_lock(&first);
/* work */
pthread_mutex_unlock(&first);
pthread_mutex_unlock(&second);
```

The two threads take the same locks in opposite orders.

If thread one gets first while thread two gets second, each then waits for the lock the other holds.

It does not happen every time. It depends on how the scheduler interleaves them, which makes it hard to find by testing.

Livelock

BLOCKED

Deadlock

Every thread in a set is blocked, waiting for an event that only another thread in the set can cause.

BUSY, WITH NO PROGRESS

Livelock

The threads are not blocked. Each keeps trying an action that fails, and tries again: like two people in a corridor who step aside in the same direction, again and again.

Livelock can be avoided by having each thread retry after a random delay.

Four necessary conditions

01

Mutual exclusion

At least one resource is held in a non-sharable mode: only one thread at a time can use it.

02

Hold and wait

A thread holds at least one resource and waits to acquire others that other threads hold.

03

No preemption

A resource is released only voluntarily, by the thread holding it, after that thread has finished its task.

04

Circular wait

T0 waits for a resource held by T1, T1 for one held by T2, and so on, until Tn waits for one held by T0.

A deadlock can arise only if all four hold at the same time. Remove any one and deadlock is impossible.

PART 2

Resource-allocation graphs

A picture of who holds what and who waits for what.

Week 11 · 09

The resource-allocation graph

Vertices

Two kinds: the threads T1, T2, … drawn as circles, and the resource types R1, R2, … drawn as rectangles with one dot for each instance.

Request edge

A directed edge from a thread to a resource type: Ti → Rj. The thread has requested an instance and is waiting for it.

Assignment edge

A directed edge from a resource instance to a thread: Rj → Ti. The instance has been allocated to that thread.

A graph with a deadlock



Following the edges brings us back to where we started: T1 → R1 → T2 → R2 → T1. Each resource has a single instance, so neither thread can ever proceed.

A cycle with no deadlock

T1

holds an instance of R2, requests R1

T3

holds an instance of R1, requests R2

R1: two instances

one held by T2, one by T3

R2: two instances

one held by T1, one by T4

T2

holds an instance of R1, waits for nothing

T4

holds an instance of R2, waits for nothing

There is a cycle: T1 → R1 → T3 → R2 → T1.

But T2 and T4 are not part of it and are not waiting.

When T4 finishes and releases its instance of R2, that instance can be given to T3. The cycle is broken.

What a cycle tells us

No cycle

No deadlock

If the graph contains no cycle, no thread in the system is deadlocked.

A cycle

One instance per type

If each resource type has exactly one instance, a cycle means deadlock.

A cycle

Several instances per type

A deadlock is possible, but not certain.

PART 3

Handling deadlocks

Three choices, and the one most systems make.

Week 11 · 14

Three ways to handle deadlocks

01

Never enter a deadlock

Use a protocol that ensures the system never becomes deadlocked: prevention or avoidance.

02

Detect and recover

Allow the system to enter a deadlocked state, detect it, and recover.

03

Ignore the problem

Pretend that deadlocks never occur. Linux and Windows do this, and leave it to the programmer to write code that does not deadlock.

Ignoring the problem is cheap. Where deadlocks are rare, the cost of preventing them all the time can be higher than the cost of an occasional restart.

Deadlock prevention

Condition to break	How	The cost
Mutual exclusion	Make resources sharable, as read-only files are	Not possible in general: a mutex lock cannot be shared
Hold and wait	A thread requests all its resources at once, or only when it holds none	Low resource utilization; starvation is possible
No preemption	A thread that cannot get a resource releases those it already holds	Does not suit locks and semaphores
Circular wait	Give every resource type a number, and always request in increasing order	The most practical of the four

Prevention makes at least one of the four necessary conditions impossible.

Breaking circular wait with an order

```
/* order: first = 1, second = 5 */

/* thread one */
pthread_mutex_lock(&first);
pthread_mutex_lock(&second);

/* thread two: the same order */
pthread_mutex_lock(&first);
pthread_mutex_lock(&second);
```

Every thread takes the locks in increasing order of their numbers.

Thread two can no longer hold second while it waits for first. A cycle cannot form.

This is the rule the opening example was missing. It works only if every programmer follows it, and it is harder when locks are chosen at run time.

PART 4

Deadlock avoidance

Use information about future requests to decide whether to grant the current one.

Week 11 · 18

The idea of avoidance

Each thread declares in advance the *maximum* number of instances of each type it may need.

What the system knows

For each thread: what it holds now and the most it could still ask for. For each type: how many instances are free.

What the system does

Before granting a request it checks that the resulting state is safe. If not, the thread waits, even though the resources are free.

The price

Resource utilization may be lower than it could be.

Safe state

Unsafe states

deadlock is possible

Deadlock

a subset of the unsafe states

Safe states

no deadlock possible

A state is safe if there is a safe sequence of all the threads.

In a safe sequence, the resources each thread may still request can be satisfied by what is free now plus what all the earlier threads in the sequence hold.

Safe: no deadlock. Unsafe: deadlock is possible, not certain. Avoidance keeps the system out of unsafe states altogether.

Two avoidance algorithms

ONE INSTANCE PER TYPE

Resource-allocation graph algorithm

For a single instance of each resource type. A claim edge, drawn dashed, says a thread may request a resource later. A request is granted only if turning it into an assignment edge creates no cycle.

SEVERAL INSTANCES

Banker's algorithm

For several instances of each type. Named after a banker who never lends out cash in a way that leaves the bank unable to satisfy all its customers.

The banker's algorithm: data

Structure	Size	Meaning
Available	vector of m	Available[j] = k: k instances of type Rj are free
Max	n × m matrix	Max[i][j] = k: thread Ti may request at most k instances of Rj
Allocation	n × m matrix	Allocation[i][j] = k: Ti currently holds k instances of Rj
Need	n × m matrix	Need[i][j] = Max[i][j] − Allocation[i][j]: what Ti may still request

n is the number of threads and m the number of resource types.

The safety algorithm

1. Let Work = Available, and Finish[i] = false for every thread.
2. Find a thread Ti with Finish[i] = false and Need[i] ≤ Work. If there is none, go to step 4.
3. Pretend Ti runs to the end and returns everything: Work = Work + Allocation[i], Finish[i] = true. Go back to step 2.
4. If Finish[i] is true for every thread, the state is safe.

The order in which threads are found in step 2 is a safe sequence. Comparing two vectors with ≤ means comparing every element.

Example: is this state safe?

ALLOCATION

	A	B	C
T0	1	0	2
T1	2	1	0
T2	3	0	1
T3	0	2	2
T4	1	1	1

MAX

	A	B	C
T0	6	3	3
T1	3	2	1
T2	5	1	4
T3	2	3	4
T4	4	2	3

NEED = MAX − ALLOCATION

	A	B	C
T0	5	3	1
T1	1	1	1
T2	2	1	3
T3	2	1	2
T4	3	1	2

Three resource types with 9, 6 and 8 instances.

Allocated in total: 7, 4, 6.

Available = (2, 2, 2).

Running the safety algorithm

Step	Thread chosen	Its Need	Work before	Work after it finishes
1	T1	(1, 1, 1)	(2, 2, 2)	(4, 3, 2)
2	T3	(2, 1, 2)	(4, 3, 2)	(4, 5, 4)
3	T2	(2, 1, 3)	(4, 5, 4)	(7, 5, 5)
4	T4	(3, 1, 2)	(7, 5, 5)	(8, 6, 6)
5	T0	(5, 3, 1)	(8, 6, 6)	(9, 6, 8)

Every thread finished, so the state is safe. A safe sequence is T1, T3, T2, T4, T0. The final Work equals the total number of instances, a useful check.

The resource-request algorithm

1. If Request[i] ≤ Need[i], continue. Otherwise it is an error: the thread has exceeded its maximum claim.
2. If Request[i] ≤ Available, continue. Otherwise Ti must wait: the resources are not free.
3. Pretend to grant it: subtract the request from Available and from Need[i], and add it to Allocation[i].
4. Run the safety algorithm. Safe: the request is granted. Unsafe: Ti waits and the old state is restored.

Two requests

GRANTED

T3 requests (1, 0, 1)

It is within T3's Need (2, 1, 2) and within Available (2, 2, 2). After the pretend grant, Available is (1, 2, 1). The safety algorithm finds T1, T3, T2, T4, T0. The request is granted.

REFUSED

T4 requests (2, 1, 2)

It is within T4's Need (3, 1, 2) and within Available. After the pretend grant, Available is (0, 1, 0). No thread's Need fits in that, not even T4's remaining (1, 0, 0). Unsafe. T4 must wait.

Both requests start from the state on the previous slides. The resources T4 asked for were free, and it was still made to wait. That is avoidance.

PART 5

Deadlock detection and recovery

Let it happen, find it, and break it.

Week 11 · 28

One instance of each type: the wait-for graph

The graph

Take the resource-allocation graph and remove the resources. An edge Ti → Tj means Ti is waiting for Tj to release a resource.

Detection

From time to time, run an algorithm that looks for a cycle. A cycle in the wait-for graph is a deadlock.

The cost

Finding a cycle takes on the order of n^2 operations, where n is the number of threads.

Several instances: the detection algorithm

1. Let Work = Available. Finish[i] = false if Ti holds any resource, and true otherwise.
2. Find a thread Ti with Finish[i] = false and Request[i] ≤ Work. If there is none, go to step 4.
3. Work = Work + Allocation[i], Finish[i] = true. Go back to step 2.
4. Every thread with Finish[i] = false is deadlocked.

It is the safety algorithm, with the current Request in place of Need. We optimistically assume a thread whose request can be met will finish and return what it holds.

Example: is there a deadlock?

ALLOCATION

	A	B	C
T0	0	1	0
T1	2	0	1
T2	3	0	2
T3	1	1	1
T4	0	1	1

REQUEST

	A	B	C
T0	0	0	0
T1	1	0	1
T2	0	0	0
T3	2	0	0
T4	0	0	2

Types with 6, 3 and 5 instances, all allocated. Available = (0, 0, 0).

T0 asks for nothing and finishes: Work = (0, 1, 0). T2 likewise: Work = (3, 1, 2). Then T1, T3 and T4 can each be satisfied in turn.

Sequence T0, T2, T1, T3, T4. No deadlock.

Now suppose T2 requests one more instance of C. After T0 finishes, Work is (0, 1, 0), and no other request fits. T1, T2, T3 and T4 are deadlocked.

When to run the detection algorithm



How often will it occur?

If deadlocks are frequent, run the algorithm frequently. Resources held by deadlocked threads stay idle, and more threads join the deadlock over time.



Two extremes

On every request that cannot be granted at once: we find the deadlock and the thread that caused it, at a high cost. Or at fixed intervals, or when CPU use drops: cheaper, but we cannot tell which thread caused it.

Recovery from deadlock

END THEM

Terminate threads

Abort all the deadlocked threads: sure, but all their work is lost. Or abort one at a time until the cycle is gone: the detection algorithm must run after each.

TAKE THEM BACK

Preempt resources

Take resources from some threads and give them to others. Three questions: which victim costs least, how to roll the victim back to a safe state, and how to make sure the same thread is not always the victim.

Choosing whom to abort: priority, how long the thread has run and how much is left, what it holds, what it still needs, and how many threads would have to end.

Key takeaways

01

Four necessary conditions

Mutual exclusion, hold and wait, no preemption and circular wait, all at once.

02

Prevention

Make one condition impossible. Ordering the locks is the practical choice.

03

Avoidance

Grant a request only if the state stays safe. The banker's algorithm checks it.

04

Detection and recovery

Look for a cycle, or run the detection algorithm, then abort or preempt.

Next session

COMING UP

Memory management

Logical and physical addresses, contiguous allocation, fragmentation, paging and swapping.

SOURCE

Reading

Silberschatz, Galvin and Gagne, *Operating System Concepts*, 10th edition, chapter 8.